

Nama : Jordan Angkawijaya

NIM : 2311102139

Laporan Praktikum Tugas Praktik Pertemuan 11

Modul 15 – Design Pattern BLoC dan Cubit pada Flutter

1. Deskripsi Aplikasi

Aplikasi TechShop adalah aplikasi daftar produk sederhana yang dibangun menggunakan Flutter dengan menerapkan pola state management BLoC/Cubit. Aplikasi ini memungkinkan pengguna untuk menjelajahi daftar produk, menambahkan produk ke keranjang, mengatur jumlah (quantity), serta melihat ringkasan pembelian secara real-time.

2. Implementasi BLoC/Cubit

2.1 CartState — Representasi State

CartState adalah kelas immutable yang merepresentasikan kondisi keranjang belanja pada satu titik waktu. Kelas ini menggunakan Equatable agar Flutter hanya melakukan rebuild ketika nilai props benar-benar berubah.

```
class CartState extends Equatable {  
  final Map<Product, int> items;      // produk → quantity  
  int get totalItems => items.values.fold(0, (s, q) => s + q);  
  double get totalPrice => items.entries  
    .fold(0, (s, e) => s + e.key.price * e.value);  
}
```

2.2 CartCubit — Logika Bisnis

CartCubit mewarisi Cubit<CartState> dan bertugas mengelola perubahan state. Setiap method membuat salinan Map yang ada, memodifikasi salinan tersebut, lalu memanggil emit() dengan state baru. Pola ini menjamin immutability.

```
void addProduct(Product product) {  
  final updated = Map<Product, int>.from(state.items);  
  updated[product] = (updated[product] ?? 0) + 1;  
  emit(state.copyWith(items: updated)); // state baru di-broadcast  
}  
void removeProduct(Product product) { /* kurangi qty atau hapus */ }  
void deleteProduct(Product product) { /* hapus langsung */ }  
void clearCart() => emit(CartState.initial());
```

2.3 BlocProvider — Penyedia State Global

BlocProvider ditempatkan di root widget (main.dart) sehingga satu instance CartCubit tersedia di seluruh widget tree. Hal ini memungkinkan ProductListScreen dan CartScreen berbagi state yang sama.

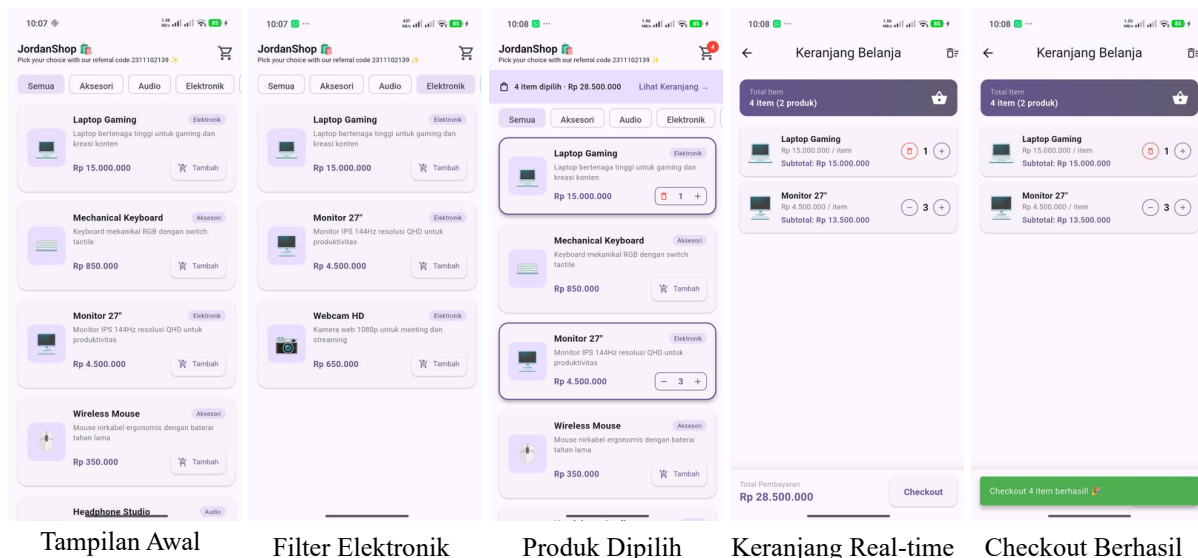
```
// main.dart  
BlocProvider(  
  create: (_) => CartCubit(),  
  child: MaterialApp(home: ProductListScreen()),  
)
```

2.4 BlocBuilder — Rebuild UI Otomatis

BlocBuilder berlangganan ke CartCubit dan memanggil ulang fungsi builder setiap kali emit() dipanggil. Digunakan di empat lokasi: badge keranjang di AppBar, strip info total item, kartu produk (tombol Tambah vs qty control), dan halaman keranjang.

```
// Badge real-time di AppBar
BlocBuilder<CartCubit, CartState>(
  builder: (context, state) {
    return Badge(
      label: Text('${state.totalItems}'),
      child: Icon(Icons.shopping_cart),
    );
  },
)
```

3. Screenshot Aplikasi



Gambar 1. Tampilan aplikasi TechShop: (1) Halaman awal saat pertama dibuka, (2) Filter kategori Elektronik aktif, (3) Produk berhasil ditambahkan ke keranjang dengan badge dan strip info terupdate, (4) Halaman Keranjang Belanja menampilkan jumlah item secara real-time, (5) Notifikasi checkout berhasil.

4. Kesimpulan

- BLoC/Cubit memisahkan logika bisnis dari UI secara tegas. CartCubit mengelola semua operasi keranjang tanpa widget perlu mengetahui cara kerjanya.
- Immutability terjaga karena setiap perubahan state menghasilkan objek CartState baru melalui copyWith(), bukan memodifikasi state yang ada.
- Efisiensi rebuild dijamin oleh Equatable — BlocBuilder hanya merender ulang widget yang terpengaruh, bukan seluruh pohon widget.
- BlocProvider sebagai dependency injection memungkinkan berbagi state lintas screen tanpa prop drilling.